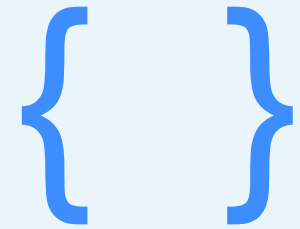


JSON in der Datenbank

OracleDB, MongoDB und MySQL
im funktionalen und praktischen Vergleich

Jonathan Kalmbach · Informatik · DHBW Horb



AUFGABE UND PROBLEMSTELLUNG

JSON ist ein Standardformat für APIs, Webanwendungen und verteilte Systeme.

OracleDB, MongoDB und MySQL unterstützen JSON direkt, verwenden dabei aber unterschiedliche Daten- und Speichermodelle.

Dadurch unterscheiden sich Abfragen, Indexierung und Änderungen trotz identischer Dokumente.

API

{ JSON }

Drei Systeme, drei technische Einbettungen

Problemstellung: Wie unterscheiden sich OracleDB, MongoDB und MySQL bei Speicherung, Abfrage, Indexierung und Änderung von JSON?

ZIELSETZUNG



Verstehen

Speicherformat, Datenmodell und JSON-Funktionen der drei Systeme einordnen.

Vergleichen

Validierung, Abfrage, Indexierung und Änderbarkeit funktional gegenüberstellen.

Messen

Typische Operationen mit identischen Testdokumenten praktisch benchmarken.

Abgrenzung

Ausgewählte Kernfunktionen und ein kontrollierter Microbenchmark, keine universelle Hersteller-Rangliste.

VORGEHEN



01



Grundlagen

Relational,
dokumentenorientiert
und JSON

02



Systemanalyse

Speichern, lesen,
indexieren und ändern

03



Benchmark

10.000 Dokumente, 5
Operationen, 2 Modi

04



Bewertung

Messwerte, Grenzen
und Einsatzszenarien

OracleDB

Tabelle → JSON-Spalte

Natives JSON wird intern als OSON gespeichert. SQL/JSON-Funktionen greifen auf Pfade zu; JSON Relational Duality bildet relationale Daten alternativ als Dokument ab.

MongoDB

Collection → BSON-Dokument

Das Dokument ist das Kernmodell. BSON repräsentiert Objekte, Arrays und zusätzliche Datentypen direkt; ein Schema ist optional validierbar.

MySQL

Tabelle → JSON-Spalte

Der native JSON-Typ validiert Eingaben und nutzt ein proprietäres binäres Format. Zugriff erfolgt mit SQL und JSON-Funktionen.

FUNKTIONALER VERGLEICH

Kriterium	OracleDB	MongoDB	MySQL
Grundmodell	Relational + JSON-Erweiterungen	Dokumentenorientiert	Relational + JSON-Erweiterungen
Speicherung	JSON-Spalte, intern OSON	Dokumente, intern BSON	JSON-Spalte, intern binär
Schema	Relationales Schema; JSON flexibel	Flexibles Schema; optionale Validierung	Relationales Schema; JSON flexibel
Abfragen	SQL/JSON und Relational Duality	Dokumentabfragen und Aggregation Pipeline	SQL/JSON und Document Store
Indexierung	JSON- und funktionsbasierte Indizes	Indizes auf Felder und verschachtelte Feldpfade, Multikey-Index	Generierte Spalten bzw. funktionale Indizes
Updates	JSON_TRANSFORM und weitere JSON-Funktionen	Operatoren wie \$set	Funktionen wie JSON_SET()

Gemeinsame JSON-Fähigkeit bedeutet nicht dasselbe Datenmodell und nicht denselben optimalen Einsatzfall.

INDEXIERUNG

MongoDB

```
create_index("status")
```

Direkter Single-Field-Index auf dem JSON-Pfad. Der Array-Pfad positionen.artikeldnummer wird automatisch zum Multikey-Index.

MySQL

```
INDEX idx_status  
(status_gen)
```

STORED Generated Column extrahiert den Wert. Darauf liegt ein normaler sekundärer B-Tree-Index; kein Array-Index.

OracleDB

```
INDEX ...  
JSON_VALUE(daten,  
'$.status')
```

Funktionsbasierter B-Tree speichert das Ergebnis von JSON_VALUE; kein Array-Index.

Vergleichbar im Zweck: status, land und bestellnummer beschleunigen. Technisch nicht identisch; die Array-Suche ist nur bei MongoDB indexiert.

BENCHMARK-DESIGN

```
{
  "bestellnummer": "B-000002",
  "status": "offen",
  "kunde": {
    "kundennummer": "K-10002",
    "name": "Kunde 2",
    "land": "US"
  },
  "positionen": [
    {
      "artikelnummer": "A-300",
      "menge": 3,
      "preis": 11.5
    },
    {
      "artikelnummer": "A-500",
      "menge": 4,
      "preis": 6.0
    }
  ],
  "zahlungsart": "PayPal",
  "erstelltAm": "2026-05-03"
}
```

10.000

synthetische Bestellungen

5

Operationen

Insert · Top-Level · verschachtelt · Array · Update

2 + 10

Warm-ups + Messläufe

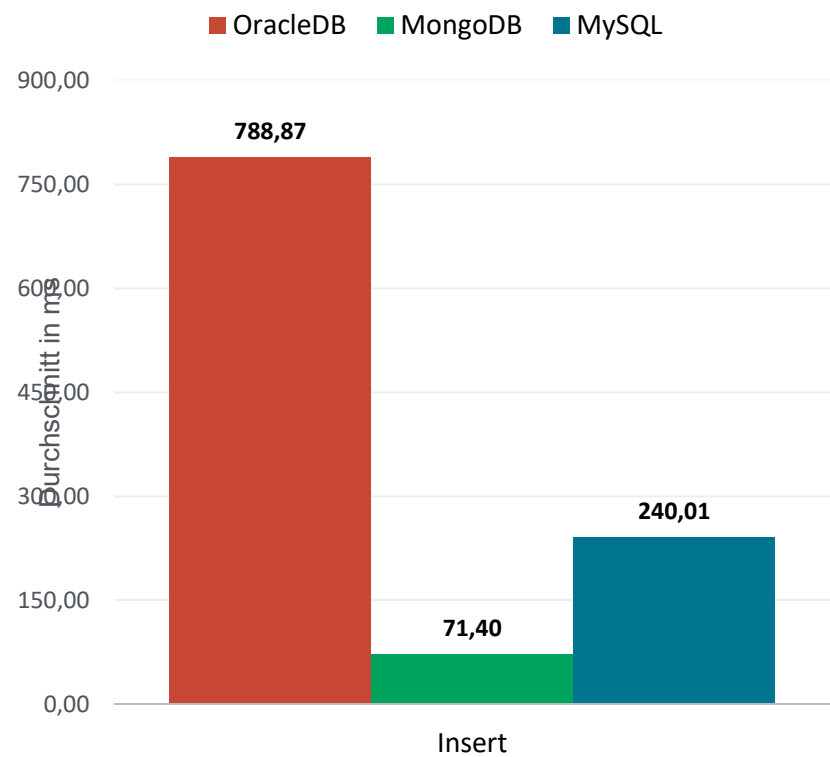
2 Modi

ohne Zusatzindex
Basic-Indexierung

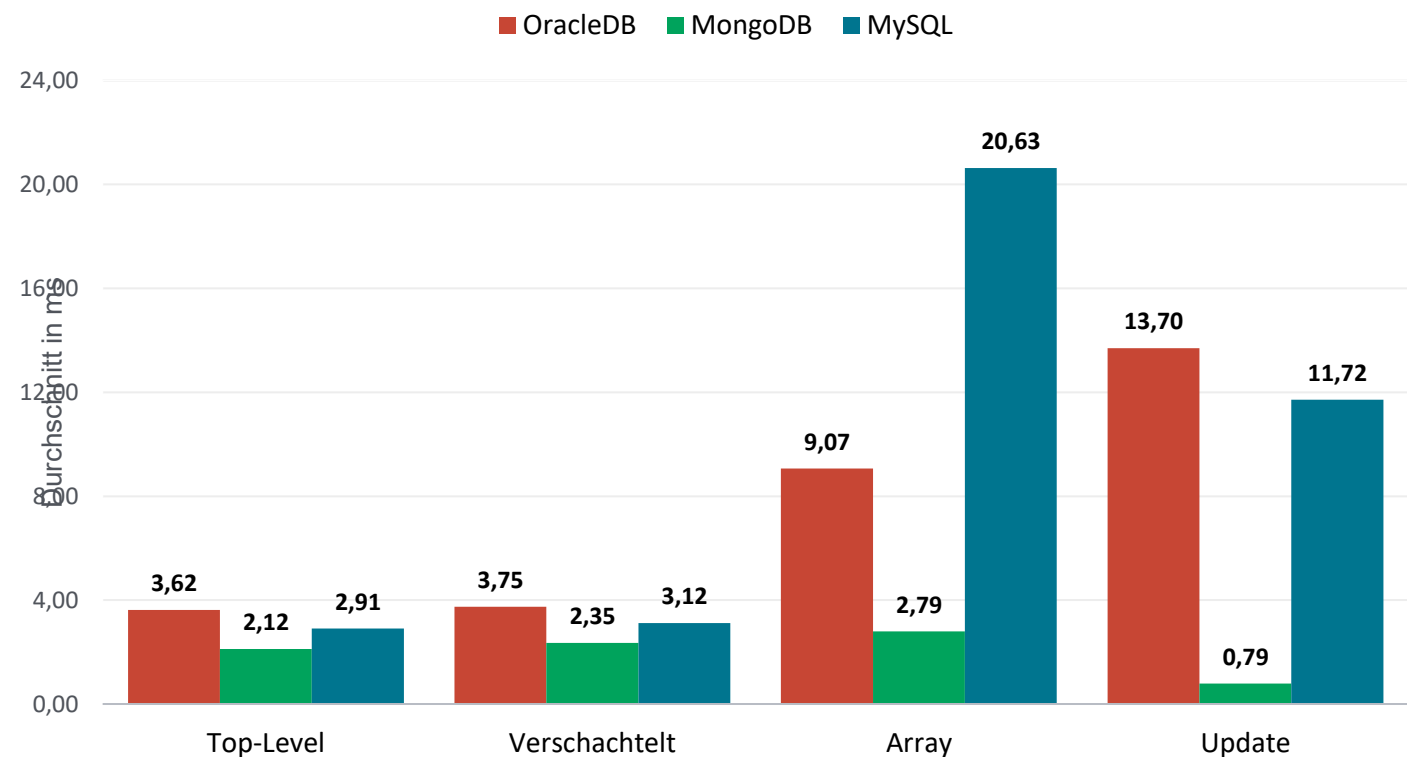
RESULTAT: OHNE INDEX



Insert



Selektionen und Update



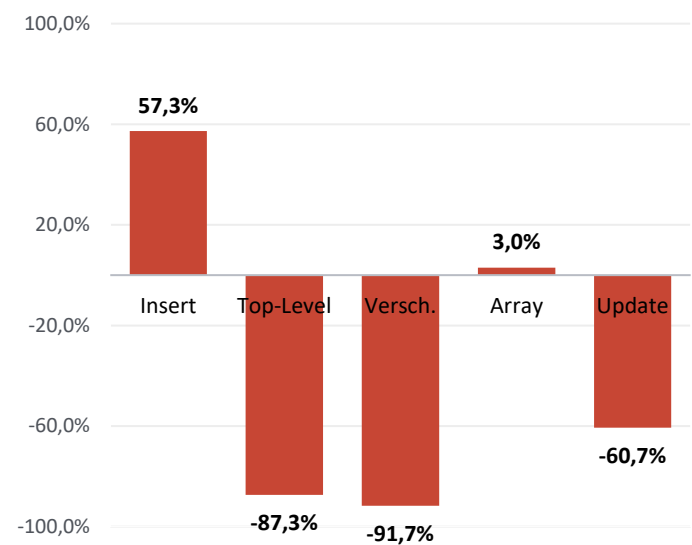
Alle fünf Operationen · Durchschnitt aus zehn Messläufen · niedriger ist besser

Quelle: Kalmbach (2026), Tab. 4 und Abb. 13; Durchschnittswerte

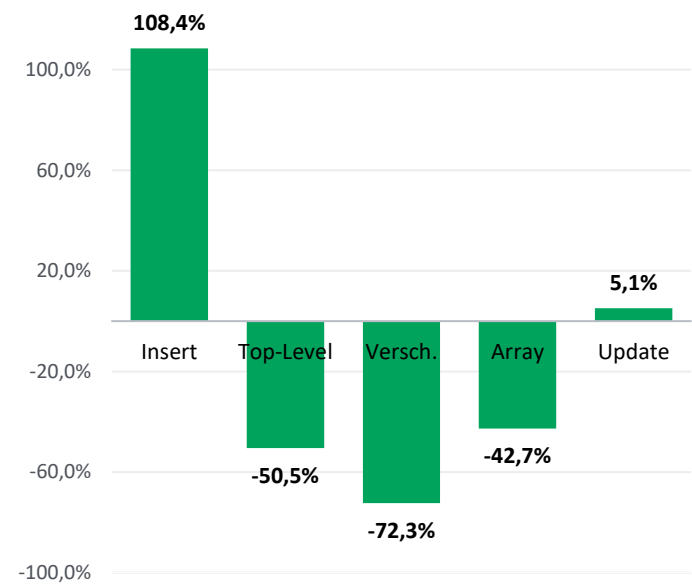
RESULTAT: INDEXWIRKUNG

Laufzeitänderung durch Basic-Indexierung in Prozent

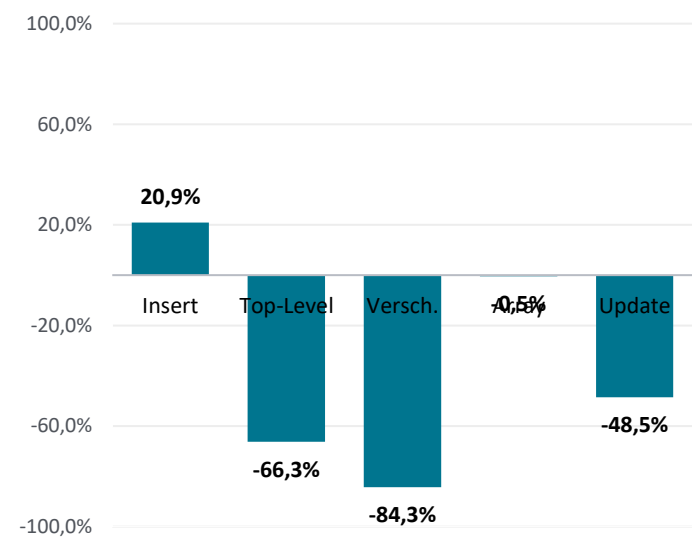
OracleDB



MongoDB



MySQL



- schneller + langsamer

Negativ = schneller, positiv = langsamer. MongoDB-Insert enthält zusätzlich die Indexerstellung; ein Array-Index wurde nur bei MongoDB verwendet.

MongoDB

Dokument zuerst

Wenn verschachtelte Dokumente das Kernmodell sind, sich Strukturen häufig ändern und Arrays direkt abgefragt werden.

OracleDB

Enterprise-Hybrid

Wenn relationale Integrität, SQL und Governance bestehen bleiben und JSON flexibel ergänzt oder per Duality bereitgestellt wird.

MySQL

Relational mit JSON

Wenn eine relationale Webanwendung einzelne flexible Attribute oder Dokumentbereiche ohne separates System benötigt.

Kein System gewinnt allgemein: Datenmodell, Zugriffspfade, Integrationsbedarf und Betriebsanforderungen bestimmen die Wahl.

Praxisteil

Benchmark live ausführen

```
python benchmark.py --rows 10000 --runs 10 --warmups 2 --index-mode basic | tee  
logs/benchmark_10000_basic.log
```

Erzeugung

10.000 identische
Testdokumente

Messung

2 Warm-ups, danach 10
gewertete Läufe

Ausgabe

Konsole, CSV und Diagramm

EINORDNUNG UND GRENZEN

Testumfang

10.000 Dokumente, eine Maschine, sequenzieller Zugriff; keine Parallelität, Skalierung oder getrennte Kaltstartmessung.

Gemessene Arbeit

Selektionen zählen Treffer, laden aber keine vollständigen Dokumente. Das Update umfasst Reset, Zielupdate und zwei Commits.

Insert-Vergleich

MongoDB misst Collection-Drop und Indexanlage im Basic-Insert mit; OracleDB und MySQL erstellen die Indexe vorher.

Indexnutzung

Die Indexe existierten, ihre konkrete Nutzung wurde jedoch nicht mit EXPLAIN, executionStats oder DBMS_XPLAN belegt.

Folgerung: Die Ergebnisse zeigen Tendenzen für das Testszenario, keine universelle Rangliste.

JSON ersetzt das relationale Modell nicht. Es erweitert es.

MongoDB behandelt Dokumente als Kernmodell und war ohne Zusatzindizes im Test am schnellsten.

Gezielte skalare Indexierung lässt OracleDB und MySQL deutlich aufholen.

Die Systemwahl beginnt bei Datenmodell und Zugriffspfaden.

Ausblick

Ausführungspläne
faire Insert-Messung
mehr Daten & Parallelität

So werden die Basic-Indizes im Skript angelegt

MongoDB

```
c.create_index("status")
c.create_index("kunde.land")
c.create_index("bestellnummer")
c.create_index(
    "positionen.artikelnummer"
)
```

Direkt, aufsteigend, nicht eindeutig;
Array-Pfad wird Multikey.

MySQL

```
status_gen VARCHAR(30)
GENERATED ALWAYS AS (
    JSON_UNQUOTE(
        JSON_EXTRACT(
            daten, '$.status')
        ) STORED,
INDEX
idx_status(status_gen)
```

Analog für land und bestellnummer;
kein Array-Index.

OracleDB

```
CREATE INDEX
idx_oracle_status
ON bestellungen_json (
    JSON_VALUE(
        daten, '$.status'
        RETURNING VARCHAR2(30)
    )
);
```

Analog für land und bestellnummer;
kein Array-Index.

Vollstaendige Durchschnittswerte des Benchmarks

Ohne Zusatzindizes

Operation	Oracle	Mongo	MySQL
Insert	788,87	71,40	240,01
Top-Level	3,62	2,12	2,91
Verschachtelt	3,75	2,35	3,12
Array	9,07	2,79	20,63
Update	13,70	0,79	11,72

Basic-Indexierung

Operation	Oracle	Mongo	MySQL
Insert	1240,71	148,82	290,16
Top-Level	0,46	1,05	0,98
Verschachtelt	0,31	0,65	0,49
Array	9,34	1,60	20,53
Update	5,39	0,83	6,03

Niedriger ist besser. Werte sind Mittelwerte aus zehn gewerteten Läufen nach zwei Warm-ups.